Exerciții C# — Liste

Nivel începător · List, while / for / foreach

mycodeschool

Pentru: Ana

Ce e nou: `List<int>`

Până acum ai lucrat cu **o singură variabilă** (`int n`) sau cu 3 obiecte pe nume (x, y, z). O **listă** ține **mai multe valori** la un loc, sub un singur nume.

```
List<int> numere = new List<int>();
numere.Add(10);
numere.Add(25);
numere.Add(4);

int cate = numere.Count;      // 3  -> câte elemente sunt
int primul = numere[0];       // 10 -> indexarea începe de la 0
int alTreilea = numere[2];    // 4
```

Cum parcurgi o listă (două variante — folosește-o pe care vrei):

```
foreach (int x in numere)
{
    Console.WriteLine(x);
}

int i = 0;
while (i < numere.Count)
{
    Console.WriteLine(numere[i]);
    i++;
}
```

> Tipar cheie: e **exact** ca la cifrele unui număr (`suma`, `contor`, `maxim`), doar că acum treci prin elementele listei, nu prin cifre.

Exercițiul 1 — Suma elementelor

Creează o listă cu câteva numere și calculează **suma** tuturor.

Exemplu (`10, 25, 4`): `Suma = 39`

> Tipar: pornești de la `int suma = 0;` și aduni fiecare element.

Exercițiul 2 — Media aritmetică

Aceeași listă. Calculează și afișează **media** elementelor.

Exemplu (`10, 25, 4`): `Media = 13.0`

> Atenție: `suma / numere.Count` e împărțire **întreagă**. Ca să ai zecimale, împarte la `(double)` — ex. `(double)suma / numere.Count`.

Exercițiul 3 — Maximul

Afișează **cel mai mare** element din listă.

Exemplu (`10, 25, 4`): `Maxim = 25`

> Tipar: presupui că primul e maximul (`int maxim = numere[0];`), apoi îl compari cu fiecare element și îl actualizezi când găsești ceva mai mare.

Exercițiul 4 — Minimul

Afișează **cel mai mic** element din listă.

Exemplu (`10, 25, 4`): `Minim = 4`

Tipar identic cu maximul, dar cu <.

Exercițiul 5 — Câte elemente sunt pare

Numără **câte** elemente din listă sunt pare.

Exemplu (`10, 25, 4`): `Elemente pare: 2`

Indiciu: contor + `if (element % 2 == 0)`.

Exercițiul 6 — Suma elementelor pare

Adună **doar** elementele pare din listă.

Exemplu (`10, 25, 4`): `Suma parelor = 14`

Exercițiul 7 — Câte sunt mai mari decât o valoare

Declară o variabilă `int prag`. Numără câte elemente sunt **strict mai mari** decât `prag`.

Exemplu (`10, 25,` 4, `prag = 9`): `Peste 9: 2`

Exercițiul 8 — Lista conține o valoare?

Declară `int cautat`. Afișează `Da` dacă valoarea se află în listă, altfel `Nu`.

Exemplu (`10, 25, 4`, `cautat = 25`): `Da`

> Indiciu: parcurgi lista; dacă găsești elementul, poți da `return` sau reține într-un `bool gasit`.

Exercițiul 9 — Câte apariții are o valoare

Declară `int cautat`. Numără de **câte ori** apare în listă.

Exemplu (`10, 4, 25, 4`, `cautat = 4`): `Apare de 2 ori`

Același tipar ca la „de câte ori apare o cifră într-un număr”.

Exercițiul 10 — Produsul elementelor

Calculează **produsul** tuturor elementelor.

Exemplu (`10, 25, 4`): `Produs = 1000`

> Atenție: contorul de produs pornește de la 1, nu de la `0`.

Exercițiul 11 — Cel mai mic și cel mai mare

Afișează **într-o singură parcurgere** (sau două) atât minimul, cât și maximul.

Exemplu (`10, 25, 4`): `Minim = 4, Maxim = 25`

Exercițiul 12 — Pozitive, negative și zero

Folosește o listă care conține și numere negative și zero. Numără câte sunt **pozitive**, câte **negative** și câte **zero**.

Exemplu (`-3, 0, 7, -1, 5`): `Pozitive: 2, Negative: 2, Zero: 1`

Indiciu: trei contoare separate + `if / else if / else`.

Exercițiul 13 — Afișare inversă

Afișează elementele listei de la **ultimul la primul**.

Exemplu (`10, 25, 4`): `4  25  10`

> Indiciu: pornești indexul de la `numere.Count - 1` și cobori până la `0`.

Exercițiul 14 — Al doilea cel mai mare

Afișează **al doilea** cel mai mare element (nu maximul, ci următorul).

Exemplu (`10, 25, 4, 25, 18`): `Al doilea maxim = 18`

> Tipar: ții **două** variabile — `maxim` și `maxim2` — și le actualizezi pe amândouă când găsești o valoare mai mare.

Recapitulare

Tip de cerință	Cum se rezolvă
Sumă / produs	contor (`0` pentru sumă, `1` pentru produs), aduni/înmulțești fiecare element
Medie	sumă împărțită la `Count`, cu `(double)` pentru zecimale
Maxim / minim	presupui primul, compari cu fiecare (> sau <)
Numărare	contor `int` crescut într-un `if`
Filtrare	`if` pe o condiție (`% 2 == 0`, `> prag`)
Căutare	parcurgi și oprești / marchezi când găsești
Parcurgere inversă	index de la `Count - 1` spre `0`